

Continuous Control & the Reality Gap

Session #2 — Intermediate

Tutorial track

RL Bootcamp 2026 · Salzburg, 16–18 September

Every number measured at repo VERSION 0.29.0

Orientation

Where this session sits — and what you leave with

Session #1 — Fundamentals

Maze worlds, Q-learning / SARSA.

The environment is a table. You wrote the algorithm.

Session #2 — you are here

MuJoCo Ant, PPO / SAC via Stable-Baselines3.

The environment is *given*. Can you tell a good result from a lucky one?

Session #3 — Advanced

Air-traffic conflict resolution, as a competition.

You design the MDP: observation, action, reward.

💡 By the end of today you can

1. Train with SB3 and **read the curves critically**.
2. Quantify the **reality gap**.
3. **Close** it — or measure what closing it cost.
4. Say what you would **need to measure** before claiming a learned policy beat anything.

⚠️ The one habit to take away

Never accept a reward number on its own.
Ask what the agent *did*.

Timetable — 2 h 30

Time	Module	Content	Hands-on
0:00–0:10	0	Orientation, environment check	—
0:10–0:25	1	Why tabular RL breaks	discussion
0:25–0:50	2	The deep RL toolkit: DQN, PG, PPO, SAC	—
0:50–1:20	3	Training in practice (SB3 + Ant)	Ex 1, 2
— 10 minute break —			
1:30–2:15	4	The reality gap, and three ways to close it	Ex 3, 4, 5
2:15–2:25	5	Configuration; the sweep runs in the background	Ex 6
2:25–2:30	—	Wrap-up, and what Session #3 asks of you	—

Nothing here needs a GPU. Small MLP policies train *faster* on CPU — the host-device transfer costs more than the matrix multiplies save.

Before anything else: does it run?

🕒 10 min

>_ Environment check — everybody, now

```
conda activate rlbootcamp
cd sessions/02-intermediate/crippled-ant

pytest                               # wrapper + config contracts, < 1 s
python -c "import sys; sys.path.insert(0, '.'); \
    from envs import make_ant; \
    e=make_ant(include_cfrc_ext_in_observation=False); \
    print(e.reset(seed=0)[0].shape, e.action_space)"
# (27,) Box(-1.0, 1.0, (8,), float32)
```

💡 If that printed

You have MuJoCo, Gymnasium, SB3 and Hydra.
Everything today runs.

⚠️ If it did not

Fix it *now*, not at exercise 3. Pair up with a neighbour — someone in the room has already solved your error.

Module 1 — Why tabular RL breaks

The Q-table does not scale

🕒 15 min

Yesterday's maze.

$|S| \times |A| = 100 \text{ cells} \times 4 \text{ moves} = \mathbf{400 \text{ entries}}$. You can print it.

Today's robot. One joint angle is a real number. Discretise it into 10 bins and the Ant's 27-dimensional observation becomes 10^{27} states — more entries than there are atoms in your body.

And the *action* is 8 continuous torques. There is no $\max_a Q(s, a)$ to take.

Two separate failures

States: you can never visit them all, so a table never generalises — two nearly identical poses are unrelated entries.

Actions: even with a finite state set, $\arg \max$ over a continuous action space is its own optimisation problem.

💡 The fix

Replace the table with a **function**: $Q(s, a; \theta)$ or $\pi(a \mid s; \theta)$, a neural network. Nearby states now share parameters — that *is* generalisation.

Where does the Maze Q-table break first — the states, or the actions?

💡 Where we are going

Whichever you argued for, the answer underneath both is the same. A table has no notion of *similar*: two states that ought to share a value are stored in separate cells, and nothing learned in one ever reaches the other.

Function approximation is not a memory trick — it is how a policy acts in a state it has never seen. **Module 4 is about what happens when that generalisation fails.**

Module 2 — The deep RL toolkit

Two families, and why only one of them fits the Ant

🕒 25 min

Value-based — learn $Q(s, a; \theta)$, act greedily.

DQN is the canonical one: experience replay breaks the correlation between consecutive transitions; a frozen target network stops you chasing a target that moves because you moved.

Policy-gradient — optimise $\pi_\theta(a | s)$ directly.

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a | s) A(s, a)]$$

$A = Q(s, a) - V(s)$: "was this action better than average *here*?" A **critic** V_ϕ supplies it — that is **actor-critic**.

⚠️ Why DQN is not on today's menu

Its output head has one unit *per action* — a discrete action space by construction.

The Ant has 8 continuous torques. There is no $\max_a Q(s, a)$ to take.

Where this lives in SB3

Actor and critic share one `MlpPolicy`.
`train/explained_variance` is exactly "is the critic any good?"

PPO and SAC — and how to pick

PPO clips the update ratio $r_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ to $[1 - \epsilon, 1 + \epsilon]$ — move too far and the objective stops rewarding you. One too-large step destroys a policy that took an hour to train, and on-policy data cannot un-destroy it.

SAC maximises reward *and* entropy, $r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))$. Exploration is paid for explicitly and α tunes itself (ent_coef: auto). Off-policy: a 1M-transition replay buffer, reused many times.

	Policy	Actions	Costs you
DQN	off	discrete	no continuous control
PPO	on	both	environment steps
SAC	off	continuous	wall-clock per step

💡 Rule of thumb

Simulator is cheap → PPO. Parallelise and let it eat steps.

Steps are precious (real robot, slow sim) → SAC.

Derivations for all four — TD loss, GAE, the clipped objective, why SAC has no `VecNormalize` — are in the appendix.

Module 3 — Training in practice

The environment: MuJoCo Ant-v5

🕒 30 min

Observation	105-dim by default: 13 torso pose + 14 joint pos/vel + 78 contact forces 27-dim with <code>include_cfrc_ext_in_observation=False</code>
--------------------	--

Action	<code>Box(-1, 1, (8,))</code> torque per joint, 2 per leg
---------------	--

Reward	forward velocity – control cost – contact cost (we zero it) + healthy bonus , +1 per step
---------------	---

Episode	<code>TimeLimit = 1000 steps</code>
----------------	-------------------------------------

Our simplification

`conf/config.yaml` drops contact forces from the observation (105 → 27) *and* the reward (`contact_cost_weight: 0.0`) — the known-good setting from the 2025 bootcamp. That term is notoriously noisy and buys nothing at this budget.

⚠️ Ant-v4 vs Ant-v5

v4 excluded contact forces and was 27-dimensional. If you copy a v4 tutorial verbatim, your observation space silently changes shape.

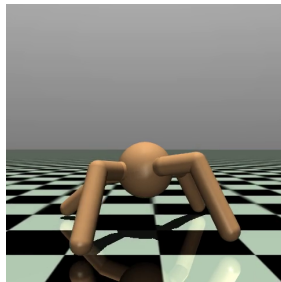
The trap that defines this session

The healthy bonus pays **+1 for every step you do not fall over**, and it is the only term you are guaranteed to collect.

So an Ant that stands perfectly still survives all 1000 steps and banks $\approx$ **1000** — measured at zero torque, **1000.6 $\pm$ 4.0**. The 1M checkpoint you would actually ship banks **987** and covers **43 cm**.

It is not a walking policy. It is a **statue**. And on a reward curve it looks like a policy that has learned something.

Everything in Module 4 depends on telling the two apart.



1M best_model: return 987, distance travelled 0.43 m



**A reward number is a sum of things you chose to pay
for.**

The statue banks 1000 of them and never moves.

Exercise 1 — your first PPO Ant

>_ Run it

```
# the 2-minute version: notebooks/01_train_ppo.ipynb, 50k steps
jupyter lab notebooks/01_train_ppo.ipynb

# the real thing (writes runs/PPO_Ant-v5/<timestamp>/)
python scripts/train.py                                # 1M steps, ~27 min
python scripts/train.py train.total_timesteps=3_000_000 # a walker, ~50 min

tensorboard --logdir runs/
```

What the script does for you

- 8 parallel envs · VecNormalize (obs + reward) ·
- EvalCallback on a *separate* env every 20k steps ·
- checkpoints with their matching normalisation stats ·
- resolved config in `.hydra/config.yaml`.

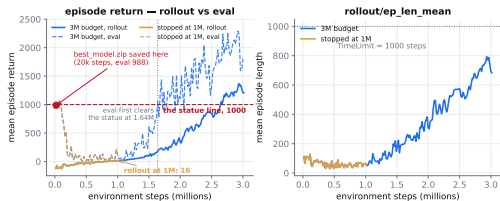
⚠ Budget reality

1M steps is *not* enough. Start the 3M run now, in a second terminal, and keep working — you will need it in Module 4.

Not waiting?

checkpoints/ppo_healthy_3M/ ships the 3M walker (1.8 MB, **2316** healthy) — it carries its own vecnormalize.pkl, and without those

Reading the curve: 1M vs 3M, same config



⚠ Stopping at 1M

At 1M the headline curve reads **16**, on episodes ending at step 65 — nowhere near the statue line. The statue is in the *other* curve: eval/mean_reward hit **988 at 20k steps**, and EvalCallback froze exactly that as `best_model.zip`. **You would ship a policy from the first 2% of the run.**

Tag

Bad sign

rollout/ep_rew_mean

flat at a round number

rollout/ep_len_mean

below the TimeLimit

eval/mean_reward

peaks early, then falls

train/entropy_loss

collapses in 50k steps

train/explained_variance ≤ 0 : predicts nothing

💡 Two curves, two different policies

rollout/* is the *sampled* policy; eval/* is the *deterministic* one, and it picks your checkpoint.

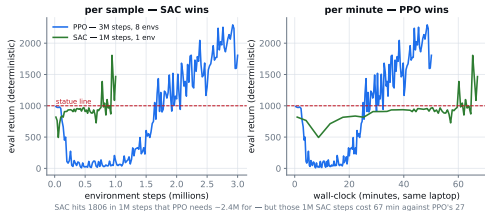
ep_len_mean is still climbing at 3M, and eval does not clear the statue line until **1.64M steps**.

Exercise 2 — PPO vs SAC: "which is better" is the wrong question

>_ Run it

```
python scripts/train.py algo=sac
```

```
# 1M steps, 1 env, ~67 min
```



💡 The two answers disagree

Per sample, SAC wins: 1806 in 1M steps; PPO needs $\approx 2.4M$ to get there.

Per minute, PPO wins: those 1M SAC steps cost 67 min. PPO does 3M in 50 min and ends at 2293.

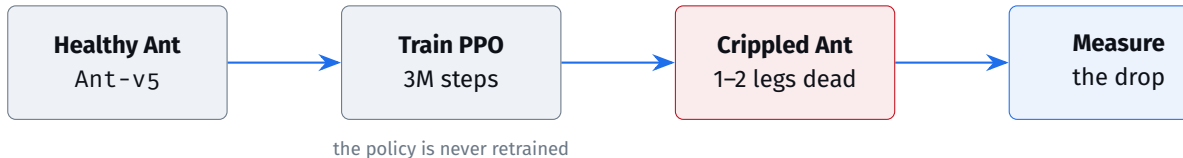
⚠ The transferable skill

Sample efficiency and wall-clock efficiency are **different axes**. Which one you optimise is a property of *your* problem, not of the algorithm. Report both, or you have not compared anything.

Module 4 — The reality gap

Train in one world, deploy in another

🕒 35 min



Why "injury" and not "noise"

A dead actuator is the cleanest domain shift there is: reproducible, parameterised by a single integer, and physically meaningful. It is also what actually happens to robots.

💡 The general lesson

Sim-to-real, distribution shift, a sensor drifting, a market regime change — all the same shape. **Your policy was fitted to a distribution, and the distribution moved.**

The CrippledAnt wrapper

```
def step(self, action):
    action = np.asarray(action, np.float32).copy()
    action[self._current_joints] = 0.0      # <-
    torque killed
    obs, reward, term, trunc, info = \
        self.env.step(action)
    info["disabled_joints"] = self.disabled_joints
    return obs, reward, term, trunc, info
```

```
make_ant(disabled_joints=[2, 3]) # exact joints
make_ant(disabled_legs=[0])      # a whole leg
make_ant(n_random_legs=1)        # new legs each
    reset
```

The wrapper zeroes **torques**, not observations. A real robot with a burnt-out motor still has working encoders — the agent sees the dead leg, it just cannot move it.

leg	position	action indices
0	front left	0, 1
1	front right	2, 3
2	back left	4, 5
3	back right	6, 7

Why we wrote our own

Off-the-shelf broken-Ant variants change the observation too, express injuries as raw joint indices, and cannot re-draw the injury on reset. All three matter here.

`info["disabled_joints"]` reports the truth every step.

Exercise 3 — the zero-shot transfer benchmark

>_ The analysis script for this module

```
# healthy, all 4 one-leg injuries, all 6 two-leg injuries – identical seeds
python scripts/transfer_benchmark.py \
    --run zero-shot=runs/PPO_Ant-v5/<3M_timestamp> \
    --model final --episodes 20 # 220 episodes, ~15-25 min

# no 3M run of your own yet? the shipped walker works the same way:
# --run zero-shot=checkpoints/ppo_healthy_3M --model final
# writes benchmark/benchmark.csv and benchmark/benchmark.png
```

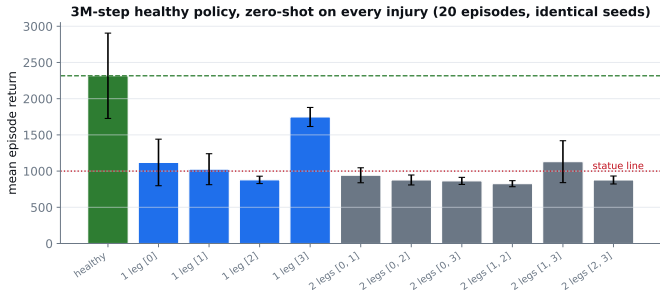
What it controls for

Episode i always uses seed `base_seed + i`, in every scenario. Every bar you compare saw the same 20 starting states, so a difference is the injury, not the draw.

⚠ Short on time?

`--episodes 5` finishes in about 5 minutes and gets you the shape of the answer. Say so in your write-up — with 5 episodes the error bars are wide enough to hide a 20% effect.

The result: half the policy gone — and not evenly



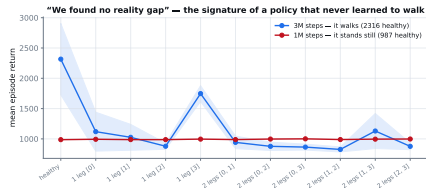
Healthy **2316** $\pm$ **589** · one leg **−48.5%** · two legs **−60.3%**

3M-step PPO · 20 seeded episodes · deterministic

💡 The bar to point at is leg 3

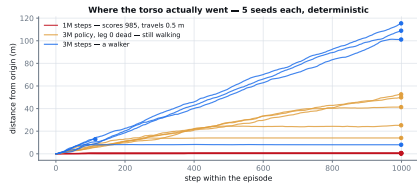
The robot is symmetric; **the gait is not**. Losing leg 2 costs **2.5×** what losing leg 3 costs — PPO converged on *one* coordination pattern and nothing asked for symmetry. **A policy is a solution, not *the* solution.**

The QA slide: "we found no reality gap"



⚠ A pair reporting this has measured a statue

A 1M-step policy scores **987 ± 2.9** healthy and **987–1000** under every injury. Standing still is injury-invariant. The flat red line is not robustness; it is the absence of behaviour.



Three behaviours the reward number cannot separate — and two blue seeds that fail (one falls at step 115, one stalls at 8 m). Never trust one seed.

💡 Three checks, ten seconds, in order

1. `ep_len_mean` approaching the TimeLimit?
2. Distance travelled $\gg 0$?
3. **Watch the render.**

And never one seed: two of the five blue tracks fail — one falls at step 115, one stalls at 8 m. That is the ± 589 in the headline number.

Exercises 4 & 5 — specialise, or randomise?

> Two more 3M policies, then one benchmark across all four

```
python scripts/train.py env.disabled_legs=[0] \  
    train.total_timesteps=3_000_000          # ~20 min  
python scripts/train.py env.n_random_legs_max=1 \  
    train.total_timesteps=3_000_000          # ~20 min  
python scripts/transfer_benchmark.py --model final --episodes 20 \  
    --run zero-shot=checkpoints/ppo_healthy_3M \  
    --run specialist=checkpoints/ppo_specialist_3M \  
    --run randomised=checkpoints/ppo_randomised_range_3M
```

4 • Specialise — fit the new point

Retrain *with* leg 0 dead, **same 3M budget** as zero-shot — a different budget would confound "less training" with "the wrong objective."

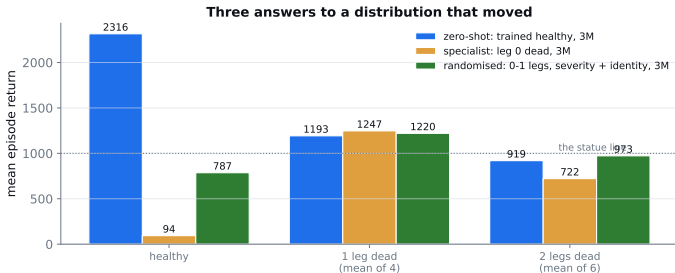
5 • Randomise — fit the distribution

Re-draw *how many* legs are dead every reset, uniform on $\{0, 1\}$, then which one: **randomise the severity too.**

⚠ Predict, then turn the page

`n_random_legs=1` (last year) never shows the policy a healthy Ant. `n_random_legs_max=1` does, half of resets — **does that recover the healthy score, or trade it away regardless?**

Robustness is bought — and 3M steps bought this much



💡 What randomising bought

On **2 legs dead** it wins outright: **973**, ahead of both the generalist (**919**) and the specialist (**722**). On **1 leg** it lands within **2%** of the specialist's own aggregate (**1220 vs 1247**) — without ever being told which leg. It never needed the failure named in advance.

⚠️ What it cost — a real cost, not a bug

Healthy: 787 — a third of the generalist's **2316**, *below the statue line*. Half its resets are healthy (unlike last year's fixed-count design), and it still does not recover healthy-specific speed.

Robustness to the injuries you randomised over is not free on the case you did not.

Module 5 — Configuration, and one honest sweep

Hydra: the config *is* the command line

🕒 10 min

```
python scripts/train.py           # defaults
python scripts/train.py algo=sac  # swap a group
python scripts/train.py env.n_random_legs=1

python scripts/train.py -m \
    algo.learning_rate=1e-4,3e-4,1e-3 \
    algo.batch_size=64,256           # -m = grid
```

Every run writes its **resolved** config to `.hydra/config.yaml`, and `evaluate.py` reads it back to rebuild the exact environment — evaluation cannot silently disagree with training. **That, not the sweep, is what this module is for.**

⚠️ Why the sweep is an exercise, not a module

6 cells $\times$ 300k steps $\approx$ 50 min — and at 300k **every cell is still a statue**. One seed per cell measures seed noise as a hyperparameter effect: one seed swings the healthy score by **500**.

Report the **ranking and its spread**, never the winner. **“1e-3 is best” you cannot earn at this budget.**

Launch it in a spare terminal during Module 4. Full walkthrough in the appendix.

Wrap-up

Five things to take to Session #3

1. **A reward number is not a result.** Report the KPIs, the episode length and a baseline next to it, or you have reported nothing.
2. **Check it walks.** `ep_len_mean`, distance travelled, then watch the render. One minute to rule out the statue; an afternoon lost if you do not.
3. **The curve you read is not the policy you ship.** `best_model.zip` came from 20k steps of a 1M run, and `rollout/ep_rew_mean` never said so.
4. **A policy is a solution.** Leg 2 costs $2.5\times$ leg 3 because nothing asked for symmetry.
5. **Beat random on every objective, or it is not priced.** On tomorrow's environment PPO is $4\times$ safer than random per step flown — and three earlier rewards produced agents *worse* than random on an objective they were supposedly optimising.

⚠ The sentence we want to hear

"It beats do-nothing and random on objectives 1 and 2, and loses to neither on any column — scored on the same 100 held-out seeds, with episode length in the table. Not because the training curve went up."

Questions?

See you in Session #3, where you get to design the reward.

Extra slides

Training hygiene — and why the wrapper is part of the model

```
venv = make_vec_env(make_ant, n_envs=8, seed=0)
venv = VecNormalize(venv, norm_obs=True,
                    norm_reward=True, gamma=0.99)

eval_venv = make_vec_env(make_ant, n_envs=1,
                        seed=10_000)
eval_venv = VecNormalize(eval_venv,
                        training=False, norm_reward=False)
```

- `make_vec_env` — 8 envs; PPO throws data away, so keep it fed. Inserts `Monitor` for you.
- `VecNormalize` — joint angles ~ 1 , velocities ~ 10 . PPO needs one scale.
- `training=False` on eval, or evaluation silently updates the statistics and your scoreboard drifts from your training env.

⚠ War story — measured on the 3M Ant, same policy

Normalisation statistics move during training, and `EvalCallback` saves `best_model.zip` **but not the statistics behind its score.**

matched statistics **2570 $\pm$ 55**

mispaired statistics **1964 $\pm$ 738**

worst episode 286 — it falls over

Not a crash. Just quietly worse, and wildly inconsistent.

Our `train.py` saves `vecnormalize.pkl` alongside every new best. `--model final` is always a matched pair.

2a — Value-based: DQN

🕒 25 min

A network approximates $Q(s, a; \theta)$; the loss is the squared TD error

$$\mathcal{L} = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

- **Experience replay.** Consecutive transitions are correlated; SGD assumes they are not. The buffer breaks the correlation and reuses data.
- **Target network θ^- .** Without a frozen copy you are chasing a target that moves because you moved.

⚠️ Why DQN is not on today's menu

The output head has one unit *per action*. That is a discrete action space by construction.

The Ant has 8 continuous torques. DQN cannot express it.

2b — Policy gradients

Skip the value function: optimise the policy directly.

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) \Psi_t]$$

What you put in Ψ_t decides the variance:

- $\Psi_t = G_t$ (Monte-Carlo return) — **REINFORCE**. Unbiased, and so noisy it is nearly unusable.
- $\Psi_t = A(s, a) = Q(s, a) - V(s)$ — the **advantage**. "Was this action better than average *here*?" Subtracting $V(s)$ changes nothing in expectation and collapses the variance.

Actor-critic

Actor $\pi_{\theta}(a | s)$ picks actions.

Critic $V_{\phi}(s)$ estimates how good the state is, so the actor gets a low-variance signal.

In SB3 both live in one `MlpPolicy` with a shared trunk — `net_arch=[256, 256]` gives each two hidden layers.

`train/explained_variance` in TensorBoard is exactly "is the critic any good?"

2c — PPO, the workhorse

The problem. A single too-large policy update destroys a policy that took an hour to train, and on-policy data cannot un-destroy it.

The fix: clip the update ratio. With

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

$$\mathcal{L} = \mathbb{E} \left[\min (r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right]$$

Move too far and the objective stops rewarding you. ϵ is `clip_range`, default 0.2.

GAE(λ) interpolates between a 1-step (biased, low variance) and Monte-Carlo (unbiased, high variance) advantage. `gae_lambda=0.95`.

💡 When to reach for PPO

Discrete *or* continuous actions, robust to hyperparameters, parallelises trivially across environments. The sane default.

⚠️ The cost

On-policy: every rollout is used for a handful of epochs and thrown away. PPO is **sample-hungry** — you pay in environment steps, which is why we run 8 envs in parallel.

2d — SAC, entropy-regularised continuous control

Maximise reward *and* randomness:

$$J(\pi) = \sum_t \mathbb{E}[r_t + \alpha \mathcal{H}(\pi(\cdot \mid s_t))]$$

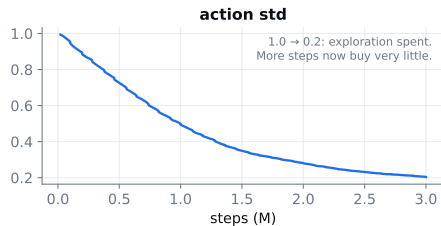
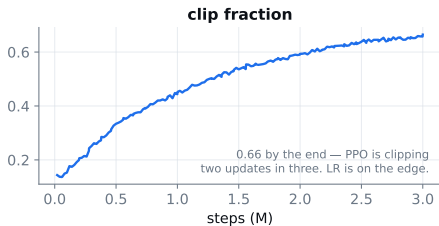
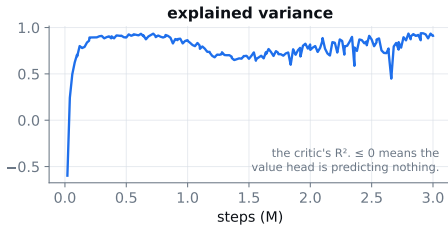
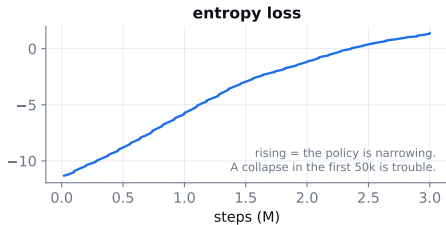
- Entropy is an explicit **exploration bonus** — the policy is paid to stay uncertain until the reward says otherwise.
- α is tuned automatically (`ent_coef: auto`); you do not hand-schedule exploration.
- **Off-policy**: a 1M-transition replay buffer, reused many times. Far fewer environment steps for the same performance.

⚠ Why our SAC config has no VecNormalize

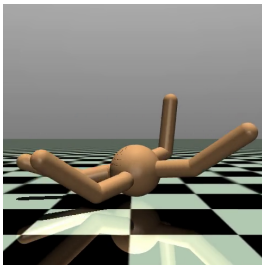
VecNormalize statistics move during training. A replay buffer would then hold transitions normalised with *stale* statistics — the same state stored under two different scalings.

The SB3 zoo does the same thing. This is a real design constraint, not an oversight.

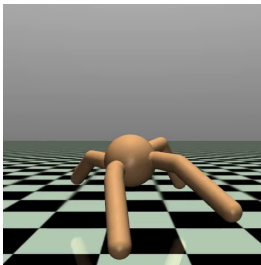
The same four tags, on the 3M run



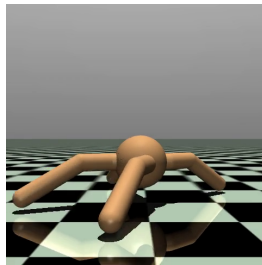
See it: the same policy, one leg at a time



healthy — 2316 [▶ play](#)



leg 0 dead — 1119 [▶ play](#)



legs 0+1 dead — 942 [▶ play](#)

>_ Render any checkpoint under any injury

```
python scripts/render_agent.py runs/PPO_Ant-v5/<ts>/best_model/best_model.zip  
python scripts/render_agent.py <zip> --disabled-legs 0 # zero-shot injury  
python scripts/render_agent.py <zip> --n-random-legs 2 --episodes 3 --stochastic
```

Command cheat sheet

Crippled Ant —

sessions/02-intermediate/crippled-ant/

```
pytest                                # contracts, <1 s
pytest -m slow                        # end-to-end, ~2 min

python scripts/train.py               # PPO healthy, 1M
python scripts/train.py algo=sac
python scripts/train.py env.disabled_legs=[0]
python scripts/train.py env.n_random_legs=1
python scripts/train.py -m +experiment=sweep_lr_batch

python scripts/evaluate.py --run <run> --model final
python scripts/evaluate.py --run <run> --disabled-legs 0
python scripts/render_agent.py <zip> --disabled-legs 0
python scripts/transfer_benchmark.py \
    --run zero-shot=<run> --episodes 20

tensorboard --logdir runs/
```

AirTraffic — sessions/03-advanced/airtraffic/

You meet this tomorrow. Left here so the sheet is complete.

```
pytest                                # env contract
python scripts/score_4d.py --agent noop --agent random
python scripts/calibrate_4d.py        # size the terms first
```

Slide assets — from the repo root

```
python slides/scripts/extract_data.py # runs/ -> slides/data/*.csv
python slides/scripts/extract_4d_board.py
python slides/scripts/make_figures.py # -> slides/figures/
python slides/scripts/make_media.py   # -> slides/media/
```

Appendix — the shipped hyperparameters

conf/algos/ppo.yaml

```
policy: MlpPolicy
policy_kwargs: {net_arch: [256, 256]}
normalize_obs: true
normalize_reward: true
learning_rate: 3.0e-4
n_steps: 2048          # x 8 envs = 16384 /
                        rollout
batch_size: 256
n_epochs: 10
gamma: 0.99
gae_lambda: 0.95
ent_coef: 0.0
clip_range: 0.2
```

conf/algos/sac.yaml

```
policy: MlpPolicy
policy_kwargs: {net_arch: [256, 256]}
normalize_obs: false   # replay + moving stats
normalize_reward: false
learning_rate: 3.0e-4
buffer_size: 1_000_000
batch_size: 256
learning_starts: 10_000
tau: 0.005
gamma: 0.99
train_freq: 1
gradient_steps: 1
ent_coef: auto
```

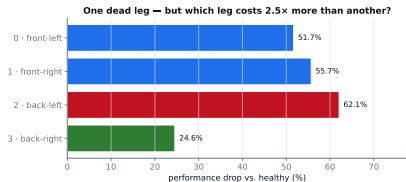
Both are close to the SB3-zoo settings for Ant. Module 5 asks which of them actually matter — do not take them on faith.

Appendix — the full transfer table

Scenario	3M policy	drop	1M policy	drop
healthy	2316 $\pm$ 589	—	987 $\pm$ 3	—
1 leg [0]	1119 $\pm$ 321	51.7%	994 $\pm$ 4	−0.6%
1 leg [1]	1026 $\pm$ 214	55.7%	988 $\pm$ 4	−0.1%
1 leg [2]	879 $\pm$ 51	62.1%	988 $\pm$ 5	−0.1%
1 leg [3]	1747 $\pm$ 132	24.6%	996 $\pm$ 4	−0.9%
2 legs [0, 1]	942 $\pm$ 104	59.3%	988 $\pm$ 3	−0.1%
2 legs [0, 2]	877 $\pm$ 69	62.1%	997 $\pm$ 5	−1.0%
2 legs [0, 3]	865 $\pm$ 49	62.7%	1001 $\pm$ 3	−1.4%
2 legs [1, 2]	826 $\pm$ 42	64.3%	989 $\pm$ 3	−0.2%
2 legs [1, 3]	1130 $\pm$ 289	51.2%	996 $\pm$ 4	−0.9%
2 legs [2, 3]	877 $\pm$ 55	62.1%	998 $\pm$ 5	−1.1%

20 episodes/cell, identical seeds, deterministic. Source `slides/data/ant_transfer.csv`. Note the standard deviations: the 3M policy is far more variable, because it is doing something.

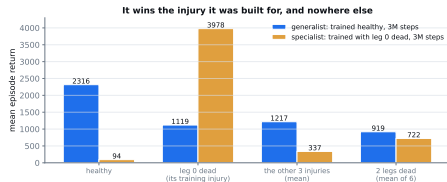
Appendix — the two transfer charts the body dropped



💡 Ask the room, then answer

The robot is symmetric — so why does losing leg 2 cost **2.5×** what losing leg 3 costs?

The gait is not. PPO converged on *one* coordination pattern, and some legs carry more of it. Nothing asked for symmetry. **A policy is a solution, not *the* solution.**



💡 Grouped by what the specialist trained on, same 3M budget

On leg 0 — the injury it was *built* for — it scores **3978** against the generalist's **1119**: a **3.6×** win. Everywhere else it loses badly: **94** healthy, **337** on the other three injuries. More training sharpened the dependency, not just the skill. Specialising pays only when the failure is known, permanent, and **detectable**.

Appendix — measured wall-clock, this laptop

What	Command	Wall-clock
contract tests	<code>pytest</code>	< 1 s
end-to-end smoke tests	<code>pytest -m slow</code>	≈ 2 min
notebook demo run (50k steps)	<code>notebooks/01_train_ppo.ipynb</code>	≈ 2 min
PPO, 1M steps, 8 envs	<code>train.py</code>	27 min
PPO, 3M steps, 8 envs	<code>train.py train.total_timesteps=3_000_000</code>	50 min
PPO, 3M steps, domain-randomised	<code>train.py env.n_random_legs=1 ...</code>	57 min
SAC, 1M steps, 1 env	<code>train.py algo=sac</code>	67 min
evaluate, 5 episodes	<code>evaluate.py --episodes 5</code>	37 s
transfer benchmark, 20 eps × 11	<code>transfer_benchmark.py</code>	15–25 min
render one episode to mp4	<code>render_agent.py</code>	≈ 1 min
sweep, 6 × 300k steps	<code>train.py -m +experiment=...</code>	≈ 50 min
AirTraffic board, 100 seeds × 2	<code>score_4d.py</code>	10 s
AirTraffic PPO, 4M steps, curriculum	<code>OMP_NUM_THREADS=1 train_4d.py</code>	≈ 80 min

16-core CPU laptop, no GPU. Halve the cores and roughly double the training numbers. Measure your own room's hardware before the session.

The package you will live in for two hours

```
sessions/02-intermediate/crippled-ant/
|- envs/crippled_ant.py          # the injury wrapper
|- scripts/
|   |- train.py                  # Hydra CLI: PPO / SAC
|   |- evaluate.py               # score a run, any
|       injury
|   |- render_agent.py           # any .zip -> .mp4
|   `-- transfer_benchmark.py    # Ex 3/4 analysis
|- conf/
|   |- config.yaml               # env + training
|       defaults
|   |- algo/{ppo,sac}.yaml
|   `-- experiment/sweep_lr_batch.yaml
|- notebooks/                    # 01 train, 02 transfer
`-- tests/                       # the definition of
    "working"
```

Two ways in.

Notebooks explode each script cell by cell — use them to *understand*.

Scripts do the bookkeeping (run dirs, checkpoints, VecNormalize, TensorBoard) — use them to *run*.

Every run lands in

`runs/<ALGO>_Ant-v5/<timestamp>/`

Monitor everything with

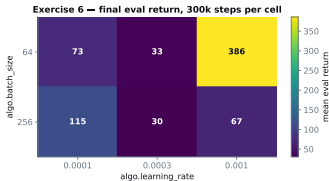
`tensorboard --logdir runs/`

Exercise 6 — the grid search

>_ Run it

```
python scripts/train.py -m +experiment=sweep_lr_batch
#   3 learning rates x 2 batch sizes = 6 runs of 300k steps
#   ~8 min per cell -> ~50 min total, sequential
#   -> runs/multirun/PPO_Ant-v5/<timestamp>/{0..5}/
```

```
python slides/scripts/extract_data.py    # aggregates the cells to CSV
python slides/scripts/make_figures.py    # -> the heatmap below
```



300k steps per cell is a *sensitivity probe*, not a tuning run — it tells you which knob the result is sensitive to, not what the best value is.

Reading a sweep honestly

⚠ Three ways a heatmap lies

- 1. One seed per cell.** You measured seed noise as if it were a hyperparameter effect. On this environment a single seed can swing the healthy score by 500.
- 2. Too short.** At 300k steps every cell is still in the statue regime. You are ranking how fast they stand up, not how well they walk.
- 3. Best-so-far vs final.** `best_eval_reward` is a maximum over a noisy series — it rewards variance.

💡 What to report instead

The **ranking** and its **spread**, not the winner:

"Learning rate moved the final score by X; batch size by Y, which is inside the seed-to-seed noise we measured. So we fixed batch size and spent the remaining budget on learning rate."

That sentence is a defensible experimental result. "1e-3 is best" is not.

Extension

Which knob matters more for the *crippled* setting than the healthy one? Re-run with `env.disabled_legs=[0]`.

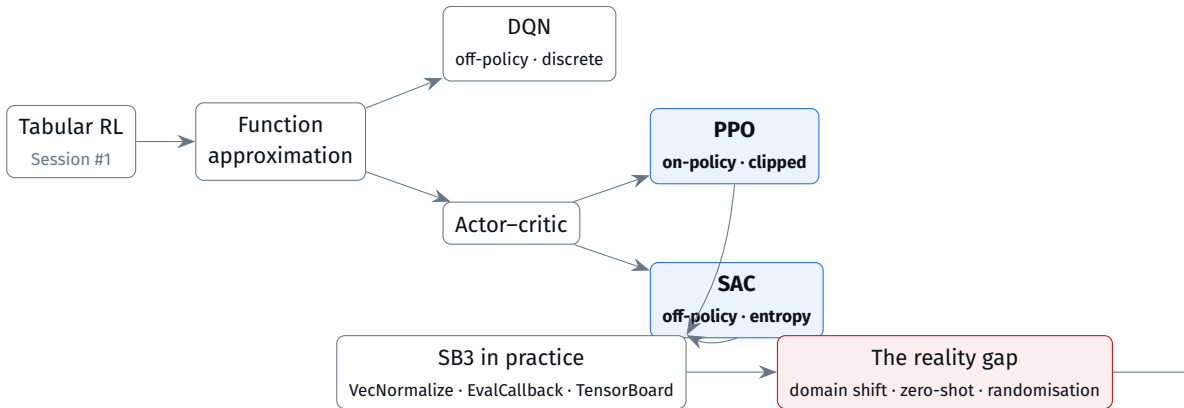
Deliverables — what you should have at the end

Ex	Deliverable	Produced by	Time
1	PPO Ant training curve	<code>train.py</code> + TensorBoard	27–50 min
2	PPO vs SAC comparison, <i>both</i> axes	<code>train.py</code> <code>algo=sac</code>	67 min
3	Healthy → crippled drop table	<code>transfer_benchmark.py</code>	15–25 min
4	Specialist vs zero-shot bar chart	<code>transfer_benchmark.py</code>	+27 min
5	Does randomisation close the gap?	<code>train.py</code> <code>env.n_random_legs=1</code>	+57 min
6	Hyperparameter heatmap	<code>train.py -m +experiment=...</code>	50 min
7	The two baselines, all three objectives	<code>score_4d.py</code>	1 min
8	Your own agent, scored against random	<code>train_4d.py</code>	25 min

⚠ The budget does not add up, and that is deliberate

Roughly four and a half hours of compute in a 3-hour session. **Launch the long runs first and work while they train**, or start Exercises 3–4 from the tracked 3M walker in checkpoints/ and spend the compute on 5 and 6. Deciding what to spend compute on is part of the skill.

The concept map



Session #3 picks up at the far right: you keep the rigour and get to design the MDP.